R26-IT-019
AI-Enabled Coconut GenBank Intelligence and Crowdsourced WCLWD Severity Assessment

component1

🌴 AI-Based Coconut Germplasm Classification System

An Artificial Intelligence-based image classification system developed to identify coconut germplasm varieties using Deep Learning and Computer Vision technologies. The system focuses on automated classification of coconut varieties such as Tall and Dwarf types using image analysis techniques.

📖 Project Overview

The Coconut Germplasm Classification System is designed to assist researchers, agricultural officers, and coconut gene banks by automatically identifying coconut germplasm varieties from images.

This component uses Artificial Intelligence, Machine Learning, and Image Processing techniques to analyze coconut morphological features and classify coconut varieties accurately.

The system helps reduce manual identification errors while improving the efficiency of germplasm conservation and agricultural research activities.

🎯 Objectives

- Automate coconut germplasm identification
- Improve classification accuracy using AI
- Support coconut gene bank management
- Reduce manual identification time
- Assist agricultural research and conservation
- Develop a smart image classification solution for agriculture

---

🧬 Coconut Germplasm Types

The system currently focuses on identifying the following coconut germplasm categories.

1️⃣ Tall Varieties

2️⃣ Dwarf Varieties

📐 Component Architecture

The system consists of the following modules.

Frontend
- Next.js
- React.js
- Tailwind CSS

Backend
- Node.js
- Express.js

AI/ML Component
- TensorFlow
- Keras
- OpenCV
- Deep Learning Models

Database
- SQLite

🤖 Deep Learning Models Used

The project compares multiple deep learning models to identify the best-performing model for coconut germplasm classification.

✅ MobileNetV2
- Lightweight CNN architecture
- Faster prediction speed
- Suitable for real-time classification

✅ EfficientNetB0
- Better feature extraction
- Higher classification accuracy
- Optimized deep learning architecture

✅ Custom CNN
- Custom-built convolutional neural network
- Used for model comparison and experimentation

🛠️ Technologies Used

Category	Technologies
Frontend	Next.js, React.js, Tailwind CSS
Backend	Node.js, Express.js
AI/ML	TensorFlow, Keras, OpenCV
Database	SQLite
Programming Languages	Python, JavaScript

| Model Training | Google Colab |

📁 Project Structure

```bash
COCONUT_GERMPLASM_CLASSIFICATION/
│
├── frontend/
│   ├── app/
│   ├── components/
│   ├── public/
│   └── package.json
│
├── backend/
│   ├── server.js
│   ├── routes/
│   ├── uploads/
│   ├── database/
│   └── package.json
│
├── python/
│   ├── predict.py
│   ├── preprocessing/
│   ├── models/
│   └── requirements.txt
│
├── dataset/
│   ├── train/
│   ├── validation/
│   └── test/
│
├── notebooks/
│   └── model_training.ipynb
│
├── README.md
└── .gitignore
```

📊 Dataset Structure

```bash
dataset/
│
├── train/
│   ├── Tall/
│   └── Dwarf/
│
├── validation/
│   ├── Tall/
│   └── Dwarf/
```

```
└── test/
    ├── Tall/
    └── Dwarf/
```

⚙️ Features

✅ Coconut Germplasm Classification
- Tall variety detection
- Dwarf variety detection
- AI-powered image classification

✅ Image Processing
- Image preprocessing using OpenCV
- Feature extraction
- Image resizing and normalization

✅ Prediction System
- Real-time image prediction
- Confidence score generation
- Automated classification workflow

✅ Dashboard Interface
- Upload coconut images
- View prediction results
- User-friendly interface
- Responsive frontend design

✅ Backend Integration
- REST API integration
- Database connectivity
- Prediction result storage

🔄 System Workflow

1. User uploads a coconut image through the frontend.
2. Frontend sends the image to the backend API.
3. Backend calls the Python prediction script.
4. AI model analyzes the uploaded image.
5. Prediction result and confidence score are generated.
6. Results are displayed on the frontend dashboard.

🚀 Installation Guide

1️⃣ Clone Repository

```bash
git clone <repository-url>
cd coconut-germplasm-classification
```

2️⃣ Setup Backend

```bash
cd backend

npm install

npm run dev
```

3️⃣ Setup Frontend

```bash
cd frontend\main-dashboard

npm install

npm run dev
```

4️⃣ Setup Python Environment

```bash
cd python

pip install -r requirements.txt

python predict.py
```

🔧 Environment Variables

Create a `.env` file inside the backend directory.

```env
PORT=5001
PYTHON_CMD=py -3.10
MODEL_PATH=models/best_coconut_germplasm_model.h5
CLASS_NAMES_PATH=models/class_names.json
```

```
---
# 📡 API Endpoints

| Method | Endpoint | Description |
|---|---|---|
| POST | `/api/predict` | Coconut germplasm prediction |
| GET | `/api/health` | Backend health status |
| GET | `/api/db-health` | Database connection status |

# 📈 Model Training

The deep learning models were trained using Google Colab with image datasets collected for Tall and Dwarf coconut varieties.

### Training Process
- Dataset collection
- Image preprocessing
- Data augmentation
- Model training
- Model evaluation
- Accuracy and loss analysis

---

# 📊 Evaluation Metrics

The following evaluation metrics were used to measure model performance.

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

---

# 🔮 Future Improvements

- Hybrid variety classification
- Larger dataset integration
- Mobile application development
- Real-time field detection
- Cloud deployment
- Grad-CAM visualization support
---

# 🌴 PP 1 - Project Status

✅ Research Phase Completed
✅ Model Training Completed
✅ Backend Development Completed
```

✅ Frontend Development In Progress
🚧 Future Enhancements Ongoing

COMPONENT 2

🌴 Crowdsourced Coconut Caterpillar Monitoring Platform

An AI-powered crowdsourced community intelligence platform developed for Coconut Caterpillar Infestation (CCI) detection, farmer knowledge sharing, and geographic pest monitoring using Deep Learning, Natural Language Processing (NLP), and data visualization technologies.

📖 Project Overview

The Crowdsourced Coconut Caterpillar Monitoring Platform is designed to assist farmers, agricultural officers, and researchers by enabling real-time pest monitoring and collaborative agricultural knowledge sharing.

The system integrates Artificial Intelligence, crowdsourced farmer observations, image analysis, NLP techniques, and geographic visualization to support early detection and monitoring of Coconut Caterpillar Infestation in coconut plantations.

The platform helps improve pest monitoring efficiency while supporting data-driven agricultural decision-making and community-assisted pest management.

🎯 Objectives

* Develop a crowdsourcing-based coconut pest monitoring platform
* Detect Coconut Caterpillar Infestation using AI-based image analysis
* Analyze farmer pest reports using NLP techniques
* Support community-driven agricultural knowledge sharing
* Visualize pest outbreak locations using geographic heatmaps
* Improve early pest detection and monitoring efficiency

🐛 Target Pest Category

The system currently focuses on monitoring the following pest category.

1️⃣ Coconut Caterpillar Infestation (CCI)

📐 Component Architecture

The system consists of the following modules.

Frontend

* React.js
* Vite
* Tailwind CSS

Backend

* Node.js
* Express.js

AI/ML Component

* TensorFlow
* Keras
* OpenCV
* CNN Transfer Learning Models

NLP Component

* spaCy
* NLTK

Database

* MongoDB

🤖 Deep Learning Models Used

The project compares multiple CNN transfer learning architectures to identify the best-performing model for Coconut Caterpillar Infestation detection.

✅ MobileNetV2

* Lightweight CNN architecture
* Faster prediction speed
* Suitable for real-time classification

✅ ResNet50

* Deep residual learning architecture
* Strong feature extraction capability
* Widely used in image classification tasks

✅ DenseNet121

* Efficient feature reuse

* Improved gradient flow
* Effective for agricultural image analysis

✅ EfficientNetB0

* Optimized CNN architecture
* Better model efficiency
* Improved classification performance

✅ InceptionV3

* Multi-scale feature extraction
* Strong image classification performance
* Effective for complex image patterns

🛠️ Technologies Used

Category	Technologies
Frontend	React.js, Vite, Tailwind CSS
Backend	Node.js, Express.js
AI/ML	TensorFlow, Keras, OpenCV
NLP	spaCy, NLTK
Database	MongoDB
Programming Languages	Python, JavaScript
Model Training	Google Colab
Data Visualization	Chart.js, Leaflet.js, Power BI

📁 Project Structure

```bash
COCONUT_CATERPILLAR_MONITORING_PLATFORM/
│
├── frontend/
│   ├── src/
│   ├── components/
│   ├── public/
│   └── package.json
│
├── backend/
│   ├── server.js
│   ├── routes/
│   ├── uploads/
│   ├── database/
│   └── package.json
│
├── python/
│   ├── predict.py
│   ├── preprocessing/
```

```
│   ├── models/
│   ├── nlp/
│   └── requirements.txt
│
├── dataset/
│   ├── train/
│   ├── validation/
│   └── test/
│
├── notebooks/
│   └── model_training.ipynb
│
├── README.md
└── .gitignore
```

📊 Dataset Structure

```bash
dataset/
│
├── train/
│   ├── CCI/
│   └── Non_CCI/
│
├── validation/
│   ├── CCI/
│   └── Non_CCI/
│
└── test/
    ├── CCI/
    └── Non_CCI/
```

⚙️ Features

✅ AI-Based Pest Detection

* Coconut Caterpillar Infestation detection
* CNN-based image classification
* Real-time image prediction

✅ Crowdsourced Pest Reporting

* Farmer pest report submission
* Coconut leaf image uploads
* Real-time field data collection

✅ NLP-Based Report Analysis

* Keyword extraction
* Entity identification
* Pest-related information analysis

✅ Community Knowledge Sharing

* Question-answer discussion platform
* Confidence-based response ranking
* Community-assisted recommendations

✅ Geographic Visualization

* Heatmap generation
* Pest outbreak monitoring
* Interactive dashboards and analytics

🔄 System Workflow

1. Farmers upload coconut leaf images and pest reports through the web platform.
2. Frontend sends the data to the backend API.
3. CNN models analyze uploaded images for suspected CCI detection.
4. NLP techniques process farmer reports and extract relevant information.
5. Community members and experts provide recommendations and discussions.
6. Confidence-based scoring ranks the most reliable responses.
7. Geographic dashboards visualize pest outbreak areas and monitoring statistics.

🚀 Installation Guide

1️⃣ Clone Repository

```bash
git clone <repository-url>
cd coconut-caterpillar-monitoring-platform
```

2️⃣ Setup Backend

```bash
cd backend

npm install

npm run dev
```

```

---

## 3️⃣ Setup Frontend

```bash
cd frontend \ maon-dashboard

npm install

npm run dev
```

4️⃣ Setup Python Environment

```bash
cd python

pip install -r requirements.txt

python predict.py
```

🔧 Environment Variables

Create a `.env` file inside the backend directory.

```env
PORT=5000
MONGODB_URI=your_mongodb_connection
JWT_SECRET=your_secret_key
MODEL_PATH=models/best_cci_model.h5
```

📡 API Endpoints

Method	Endpoint	Description
POST	`/api/predict`	CCI image prediction
POST	`/api/report`	Submit pest report
GET	`/api/heatmap`	Retrieve pest heatmap data
GET	`/api/discussions`	Community discussion data

📈 Model Training
```

The CNN models were trained using Google Colab with coconut leaf image datasets for CCI and Non-CCI classification.

Training Process

* Dataset preparation
* Image preprocessing
* Data augmentation
* Dataset splitting
* CNN transfer learning
* Model evaluation
* Accuracy and loss analysis

📊 Evaluation Metrics

The following evaluation metrics were used to measure model performance.

* Accuracy
* Precision
* Recall
* F1-Score
* Confusion Matrix

🔒 Validation & Reliability Mechanisms

* AI-based CNN filtering for uploaded images
* NLP-based farmer report analysis
* Confidence-based response ranking
* Community voting and repeated solution validation
* Expert/admin verification support

🔮 Future Improvements

* Multi-pest detection support
* Mobile application development
* Real-time field detection
* Cloud deployment
* Advanced explainable AI visualizations
* IoT-based agricultural monitoring integration

COMPONENT 3

🌴 GIS-Based Coconut Planting Spot Detection Component

An OpenCV-based GIS support component developed to analyze drone or satellite images of coconut plantation land and recommend suitable coconut planting spots by detecting canopy areas, obstacles, plantable gaps, and spacing-based planting locations.

📖 Component Overview

The GIS-Based Coconut Planting Spot Detection Component is designed to support coconut plantation planning by identifying suitable areas for new coconut planting using aerial imagery.

The system analyzes top-down drone or satellite images of coconut plantation land. It detects existing coconut canopy, identifies obstacles such as roads, buildings, soil paths, and non-plantable regions, and highlights suitable plantable areas. Based on the selected spacing value, the system generates recommended planting spot locations and produces a visual output image.

This component helps agricultural officers, researchers, and plantation planners make better land-use decisions using image processing techniques.

🎯 Objectives

* Analyze drone or satellite images of coconut plantation land
* Detect existing coconut canopy areas
* Identify obstacles and non-plantable regions
* Detect open gaps and suitable plantable zones
* Generate recommended coconut planting spots using spacing control
* Produce visual output showing plantable zones, obstacles, and planting locations

---

🛰️ Input Data

The system uses top-down aerial images such as:

* Drone images of coconut plantations
* Satellite images of coconut plantation land
* GIS-based land images with visible coconut canopy and open gaps

Supported image formats:

* JPG
* JPEG
* PNG
* WebP

🏗️ Component Architecture

The component consists of the following main parts.

Frontend

* HTML
* CSS
* JavaScript

Backend

* Python
* Flask
* Flask-CORS

Image Processing

* OpenCV
* NumPy

Storage

* Local upload folder for input images
* Local output folder for processed result images

🛠️ Technologies Used

Category	Technologies Used
Programming Language	Python, JavaScript
Backend Framework	Flask
Frontend	HTML, CSS, JavaScript
Image Processing	OpenCV
Numerical Processing	NumPy
Environment Handling	python-dotenv
API Communication	Flask-CORS
Data Source	Drone/Satellite Images

🧠 Algorithm Used

The system uses a Canopy Anomaly Detection approach with OpenCV image processing.

Main Algorithm Steps

1. Convert the uploaded image into HSV and grayscale formats.

2. Detect dense coconut canopy using HSV color thresholding.
3. Treat non-canopy regions as anomaly regions.
4. Remove annotation marks and border artifacts.
5. Detect obstacles using color, shape, brightness, texture, and contour analysis.
6. Detect plantable zones such as open lawn, green gaps, and bare soil patches.
7. Remove existing canopy and obstacles from plantable areas.
8. Generate planting spots using spacing-based grid placement.
9. Draw the final output image with plantable zones, obstacles, and planting spots.

🌿 Canopy Detection

Coconut canopy refers to the visible top leafy cover of coconut trees in aerial images.

The system identifies canopy using HSV thresholding. Dense coconut canopy is usually dark and saturated green in drone or satellite images. After detection, morphological opening is used to remove noise and keep stronger canopy regions.

Canopy areas are removed from plantable zones because new planting spots should not overlap with existing coconut trees.

🚧 Obstacle Detection

The system detects non-plantable areas such as:

* Buildings
* Roofs
* Dirt roads
* Soil paths
* Structural clearings
* Non-green land regions
* Other obstacles inside plantation land

Obstacle detection is performed using:

* HSV thresholding
* Canopy anomaly detection
* Morphological operations
* Sobel edge detection
* Contour analysis
* Shape and texture filtering

Detected obstacles are shown in red in the output image.

🌱 Plantable Zone Detection

Plantable zones are areas where new coconut trees can be planted.

The system detects:

* Open lawn areas
* Gaps between existing coconut trees
* Suitable green land areas
* Bare soil areas

Existing canopy and obstacles are removed before generating planting spots.

📍 Planting Spot Generation

Planting spots are generated using spacing-based grid placement.

The spacing value controls the distance between planting spots.

Example:

* Higher spacing → fewer planting spots
* Lower spacing → more planting spots

The system ensures that planting spots are placed only inside valid plantable zones and not on obstacles or existing coconut canopy.

🔄 System Workflow

1. User uploads a drone or satellite image through the frontend.
2. Flask backend receives the uploaded image.
3. The system validates the image format and image suitability.
4. OpenCV reads and converts the image into HSV and grayscale.
5. Coconut canopy areas are detected.
6. Obstacle regions are identified using canopy anomaly analysis.
7. Plantable zones are detected by excluding canopy and obstacles.
8. Spacing-based planting spots are generated.
9. Output image is drawn with color-coded results.
10. Backend returns the processed image and result details to the frontend.

🎨 Output Result

The final output image displays:

| Output Element | Meaning |

| ------------- | ------- |
Green Areas	Plantable zones
Red Areas	Obstacles / non-plantable areas
Teal Numbered Circles	Recommended planting spots

The backend also returns:

* Output image filename
* Number of planting spots
* Number of plantable zones
* Number of detected obstacles
* X and Y coordinates of planting spots

📡 API Endpoint

Method	Endpoint	Description
POST	`/analyze`	Upload image and analyze planting spots
GET	`/outputs/<filename>`	Retrieve generated output image
GET	`/`	Load frontend page

📁 Project Structure

```bash
GIS_COCONUT_PLANTING_SPOT_DETECTOR/
│
├── app.py
├── templates/
│   └── index.html
├── uploads/
│   └── uploaded images
├── outputs/
│   └── processed result images
├── requirements.txt
├── .env
└── README.md
```

⚙️ Installation Guide

1️⃣ Clone Repository

```bash
git clone <repository-url>
cd gis-coconut-planting-spot-detector
```

2️⃣ Create Virtual Environment

```bash
python -m venv venv
venv\Scripts\activate
```

3️⃣ Install Dependencies

```bash
pip install flask flask-cors python-dotenv opencv-python numpy
```

Or install using requirements file:

```bash
pip install -r requirements.txt
```

4️⃣ Run the Application

```bash
python app.py
```

Open in browser:

```bash
http://127.0.0.1:5000
```

🔧 Environment Variables

Create a `.env` file in the project root.

```env
PORT=5000
UPLOAD_FOLDER=uploads
OUTPUT_FOLDER=outputs
DEFAULT_SPACING=52
MIN_SPACING=30
MAX_SPACING=100
FLASK_DEBUG=true
```

✅ Validation Mechanisms

The system validates uploaded images by checking:

* Image readability
* Image size
* Supported file type
* Vegetation presence
* Coconut/palm canopy visibility
* Top-down aerial/satellite view suitability
* Usable land content

Invalid images are rejected with clear error messages.

📊 Evaluation / Testing Approach

The component can be tested using different drone and satellite images by checking:

* Correct canopy detection
* Correct obstacle detection
* Correct plantable zone identification
* Correct planting spot placement
* Output image clarity
* Response accuracy from backend API

🔒 Reliability Features

* Rejects non-aerial images
* Removes image border artifacts
* Ignores previous output annotations
* Uses multiple plantable detection strategies
* Uses fallback logic for low-contrast images
* Prevents planting spots from overlapping obstacles and canopy

---

🔮 Future Improvements

* Integration with real GIS map layers
* Support for georeferenced drone images
* Database storage for analysis history
* Officer verification dashboard
* Improved field boundary detection
* Advanced plantation density analysis
* Mobile-based image upload support

🌴 Project Status

✅ Image upload implemented
✅ Image validation implemented
✅ Canopy detection implemented
✅ Obstacle detection implemented
✅ Plantable zone detection implemented
✅ Planting spot generation implemented
🚧 GIS layer integration planned
🚧 Database storage planned

COMPONENT 4--

🌴 Weligama Coconut Leaf Wilt Disease Detection and Severity Assessment Using Vision Transformer

---

📖 Project Overview

This research focuses on developing an AI-based system for detecting Weligama Coconut Leaf Wilt Disease (WCLWD) using deep learning and Vision Transformer techniques. The system classifies coconut leaf disease images while improving explainability through Grad-CAM and Vision Transformer attention visualization.

The research compares multiple CNN architectures with Vision Transformer models to analyze classification performance, localization capability, and contextual feature learning. The system is further integrated with the backend to support real-time disease prediction.

🎯 Objectives

- Detect WCLWD symptoms using deep learning models
- Compare CNN architectures with Vision Transformer
- Improve explainability using Grad-CAM and attention visualization
- Support automated and early disease identification
- Enable future disease severity assessment

🌿 Target Disease Categories

The dataset currently contains the following WCLWD disease categories.

1️⃣ WCLWD Drying of Leaflets
2️⃣ WCLWD Flaccidity
3️⃣ WCLWD Yellowing

📐 Component Architecture

Frontend
- React.js
- Vite
- Tailwind CSS

Backend
- Node.js
- Express.js

AI/ML Component
- PyTorch
- Torchvision
- OpenCV
- Vision Transformer (ViT)
- CNN Transfer Learning Models

Database
- MongoDB

🤖 Deep Learning Models Used

✅ Custom CNN
- Baseline CNN model
- Basic convolution feature extraction

✅ VGG16
- Deep CNN architecture
- Strong feature extraction capability

✅ ResNet50
- Residual learning architecture
- Grad-CAM explainability support

✅ InceptionV3
- Multi-scale feature extraction
- Improved spatial learning

✅ EfficientNet-B0
- Efficient model scaling
- Balanced performance and efficiency

✅ Vision Transformer (ViT-B/16)

- Transformer-based image classification
- Global contextual feature learning
- Attention-based localization

🛠️ Technologies Used

Category	Technologies
Frontend	React.js, Vite, Tailwind CSS
Backend	Node.js, Express.js
AI/ML	PyTorch, Torchvision, OpenCV
Database	MongoDB
Programming Languages	Python, JavaScript
Model Training	Google Colab
Explainability	Grad-CAM, Attention Visualization

📂 Project Structure

```bash
WCLWD_DETECTION_SYSTEM/
│
├── frontend/
├── backend/
├── python/
│   ├── models/
│   ├── preprocessing/
│   ├── predict.py
│   └── requirements.txt
│
├── dataset/
│   ├── train/
│   └── test/
│
├── notebooks/
│   └── model_training.ipynb
│
├── README.md
└── .gitignore
```

📊 Dataset Information

Disease Category	Image Count
WCLWD Drying of Leaflets	1078
WCLWD Flaccidity	1069
WCLWD Yellowing	1084

Total Dataset Size: Approximately 3231 images

⚙️ Features

✅ AI-Based Disease Detection
- Coconut leaf disease classification
- Real-time image prediction
- Multiple model comparison

✅ Explainable AI
- Grad-CAM visualization
- Vision Transformer attention maps
- Disease localization analysis

✅ Backend Integration
- Real-time prediction support
- Model integration with backend API

🔄 System Workflow

1. Coconut leaf images are collected and organized.
2. Images are preprocessed and augmented.
3. Dataset is split into training and testing sets.
4. CNN models and Vision Transformer are trained.
5. Models are evaluated using performance metrics.
6. Grad-CAM and attention visualization are generated.
7. Backend predicts disease classes from uploaded images.

📈 Model Training Process

The models were trained using Google Colab.

Training Steps
- Dataset preparation
- Image preprocessing
- Data augmentation
- Train-test splitting
- Transfer learning
- Model evaluation
- Explainability analysis

📊 Evaluation Metrics

The following metrics were used for performance evaluation.

- Accuracy
- Precision
- Recall
- F1-score

🔒 Validation Techniques

- 80:20 train-test split
- Evaluation using unseen test images
- Comparative analysis across multiple architectures
- Explainability validation using Grad-CAM and attention maps

---

🔮 Future Improvements

- Weakly Supervised Semantic Segmentation (WSSS)
- Disease Severity Index (DSI) calculation
- ABCD feature extraction
- Real-time deployment optimization
- Expanded real-world dataset collection

👩‍💻 Developed By

MALSHANI W.A.M.S
SLIIT Faculty of Computing

📄 License

This project is developed for academic and research purposes.

🌴 Project Status

✅ Dataset Preparation Completed
✅ CNN Model Training Completed
✅ Vision Transformer Implementation Completed
✅ Grad-CAM Visualization Completed
✅ Attention Map Visualization Completed
✅ Backend Integration Completed
🚧 Severity Assessment Enhancements Ongoing

Conclusion

This project presents an AI-powered smart coconut research and plantation support system that integrates four major components: coconut germplasm identification, GIS-driven land utilization analysis, crowdsourcing-based community intelligence, and WCLWD severity assessment. By combining artificial intelligence, image processing, spatial analysis, and community-driven data collection, the system aims to support farmers, researchers, and agricultural authorities in improving coconut plantation management, disease monitoring, and germplasm conservation. The project demonstrates the potential of modern AI technologies to contribute toward smarter and more sustainable agricultural practices in Sri Lanka.